



LocaleGuide

**Plateforme de Local Guide
pour Monastir Centre**

Présenté par :

Mejri Maram
Werghemmi Isra
Mejbri Tasnim
Louati Souha

Enseignante :

Senda Laatawi

Établissement :

Institut Supérieur d'Informatique et de Mathématiques
Monastir

Année universitaire :

2025–2026

Table des matières

1	Introduction et présentation de l'idée	2
1.1	Contexte et objectifs	2
1.2	L'idée originale	2
1.3	Défense de l'idée (justification)	2
2	Conception	4
2.1	Diagramme des cas d'utilisation	4
2.2	Détail des cas d'utilisation	5
2.2.1	Détail des cas d'utilisation (UC détaillés)	5
2.3	Diagrammes de séquence	6
2.3.1	Séquence : Réserver un guide	6
2.3.2	Séquence : Authentification et création de profil	6
2.4	Diagramme entité-association (modèle de données)	6
2.5	Architecture technique	7
2.6	Maquettes / wireframes	7
3	Développement	12
3.1	Technologies choisies et justifications	12
3.2	Organisation du code	12
3.3	Implémentation des fonctionnalités clés	14
3.3.1	Authentification et gestion des rôles	14
3.3.2	Recherche de commerces	15
3.3.3	Gestion des guides	15
3.3.4	Upload de la photo du guide	16
3.3.5	Sécurité Row Level Security	17
4	Tests et validation	18
4.1	Environnement et outils de test	18
4.2	Scénarios de test	19
5	Conclusion et perspectives	20
5.1	Bilan du projet	20
5.2	Difficultés rencontrées et apprentissages	20
5.3	Évolutions possibles	20
6	Annexes	21
6.1	Lien vers le dépôt Git	21
6.1.1	Fichiers de configuration	22
6.1.2	Fichier d'environnement (.env)	23

1 Introduction et présentation de l'idée

1.1 Contexte et objectifs

Dans un contexte de digitalisation du secteur touristique, les utilisateurs recherchent des solutions numériques fiables leur permettant de découvrir facilement des commerces locaux, des guides touristiques ainsi que des événements.

Le projet **LocalGuide Monastir** s'inscrit dans cette perspective en proposant une application web moderne permettant de centraliser ces services sur une seule plateforme.

Les objectifs principaux du projet sont :

- Faciliter la découverte des commerces locaux
- Mettre en relation les utilisateurs avec des guides touristiques
- Permettre la consultation et la publication d'avis
- Offrir un système de réservation simple et efficace

1.2 L'idée originale

L'idée principale du projet consiste à développer une plateforme interactive qui connecte différents acteurs du domaine touristique : les utilisateurs, les commerces locaux et les guides.

L'application permet :

- La recherche intelligente de commerces et services
- L'ajout de nouveaux commerces par les utilisateurs
- La candidature pour devenir guide touristique
- La gestion dynamique des contenus via une base de données

Cette solution apporte une valeur ajoutée en centralisant les informations et en améliorant l'expérience utilisateur.

- **Découvrir** les commerces locaux (restaurants, cafés, boutiques, artisans) avec leurs horaires, prix et avis clients.
- **Réserver** des guides touristiques certifiés, spécialisés dans différents thèmes (gastronomie, histoire, mer, artisanat, nocturne).
- **Suivre** les actualités locales : événements, promotions, nouvelles ouvertures.
- **Contribuer** à la communauté en laissant des avis ou en proposant ses services de guide.

1.3 Défense de l'idée (justification)

Le choix de ce projet repose sur plusieurs raisons :

- **Utilité réelle** : répondre aux besoins des touristes et des habitants
- **Innovation** : intégration de fonctionnalités modernes (recherche avancée, avis, réservation)
- **Accessibilité** : interface simple et intuitive
- **Impact local** : valorisation des commerces et guides locaux

Ce projet constitue ainsi une solution pertinente, à la fois utile et évolutive, dans le domaine du tourisme numérique.

2 Conception

2.1 Diagramme des cas d'utilisation

Diagramme des cas d'utilisation principal

Le système **LocalGuide** implique quatre acteurs principaux :

- **Visiteur** (non connecté) : peut consulter les commerces, les guides et les actualités.
- **Utilisateur** (connecté) : peut laisser des avis, réserver des guides et des commerces.
- **Candidat Guide** : peut soumettre une demande pour devenir guide.
- **Administrateur** : modère les avis, valide les guides et les commerces, et gère toutes les actualités.

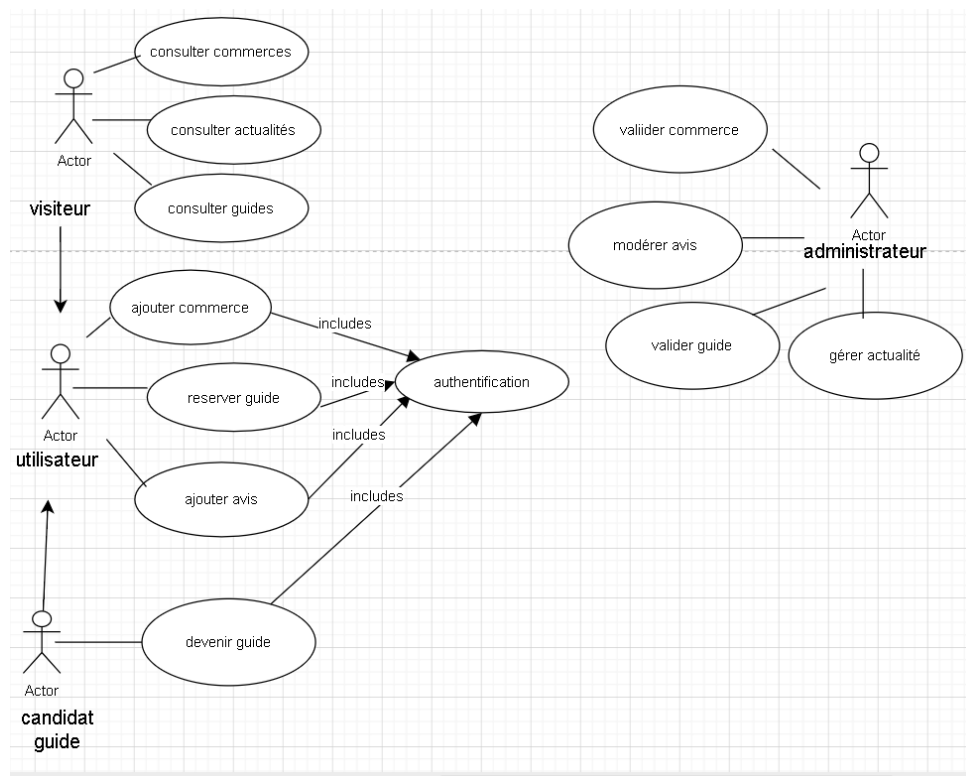


FIGURE 2.1 – le diagramme des cas d'utilisation du système LocalGuide.

La figure ?? présente le diagramme des cas d'utilisation du système LocalGuide.

2.2 Détail des cas d'utilisation

2.2.1 Détail des cas d'utilisation (UC détaillés)

UC1 : Ajouter un commerce

Acteur principal : Utilisateur

Préconditions :

— L'utilisateur est authentifié

Postconditions :

— Le commerce est enregistré dans la base avec statut *pending* (en attente de validation admin)

Scénario nominal :

1. L'utilisateur accède à la page "Ajouter un commerce"
2. Il remplit les informations (nom, catégorie, description, adresse, image, etc.)
3. Il soumet le formulaire
4. Le système envoie les données au backend (Supabase)
5. Le commerce est enregistré dans la table `commerces`
6. Un message de confirmation s'affiche

Scénarios alternatifs :

- Si un champ obligatoire est vide → message d'erreur
- Si erreur serveur → message "échec d'ajout"

UC2 : Devenir guide

Acteur principal : Utilisateur

Préconditions :

— L'utilisateur est authentifié

Postconditions :

— Une demande de guide est enregistrée avec statut *pending*

Scénario nominal :

1. L'utilisateur accède à la page "Devenir guide"
2. Il remplit le formulaire (nom, spécialité, langues, expérience, etc.)
3. Il télécharge sa photo
4. Il valide la demande
5. Le système envoie les données au backend
6. Les données sont insérées dans la table `guides`
7. Un message de confirmation s'affiche

Scénarios alternatifs :

- Si les données sont invalides → message d'erreur
- Si l'upload de l'image échoue → demande de réessayer

2.3 Diagrammes de séquence

2.3.1 Séquence : Réserver un guide

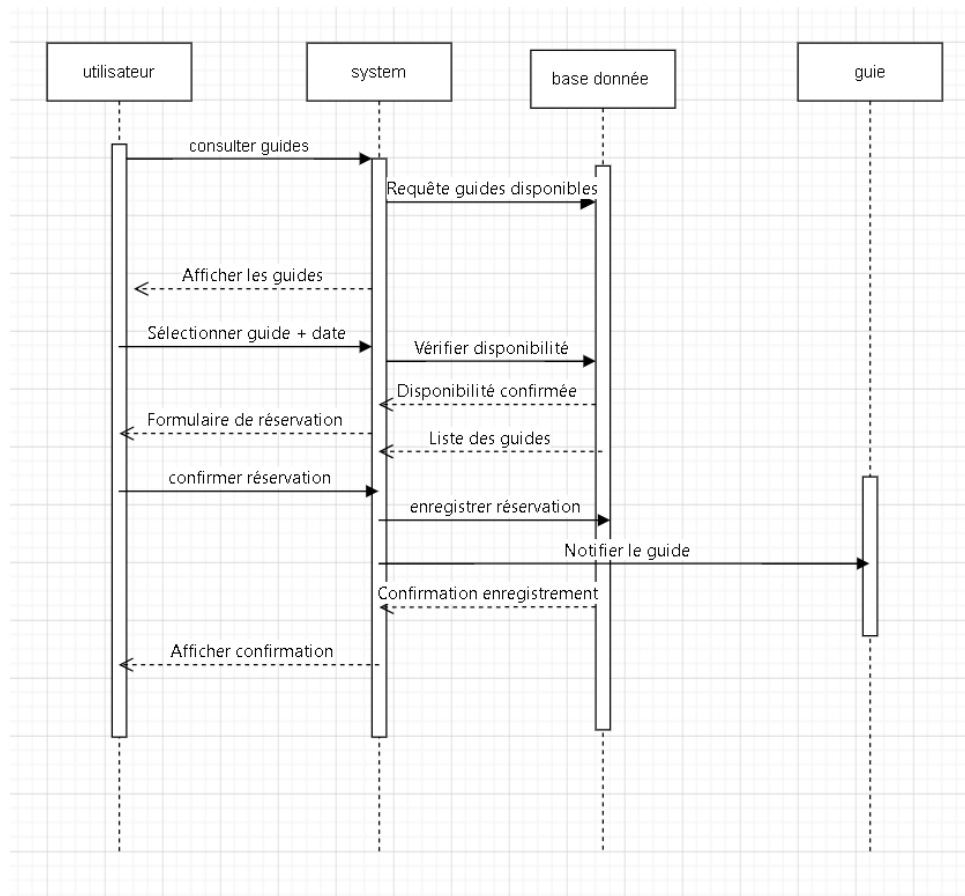


FIGURE 2.2 – Diagramme de séquence - Réservation d'un guide

2.3.2 Séquence : Authentification et création de profil

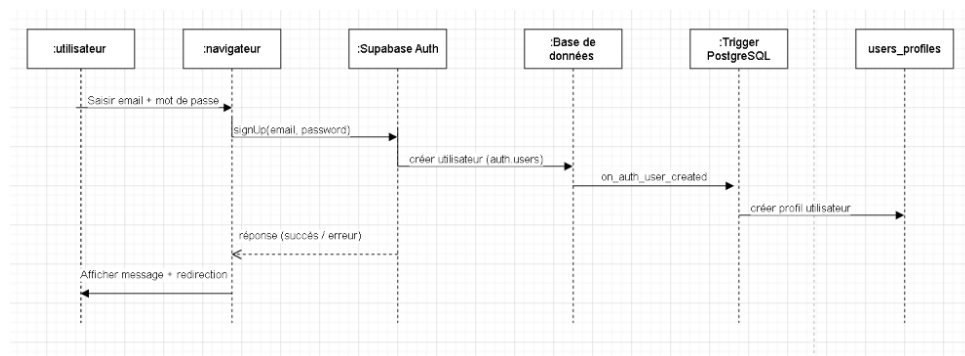


FIGURE 2.3 – Diagramme de séquence - Authentification et création de profil

2.4 Diagramme entité-association (modèle de données)

Le schéma relationnel de LocalGuide s'articule autour de six tables principales :

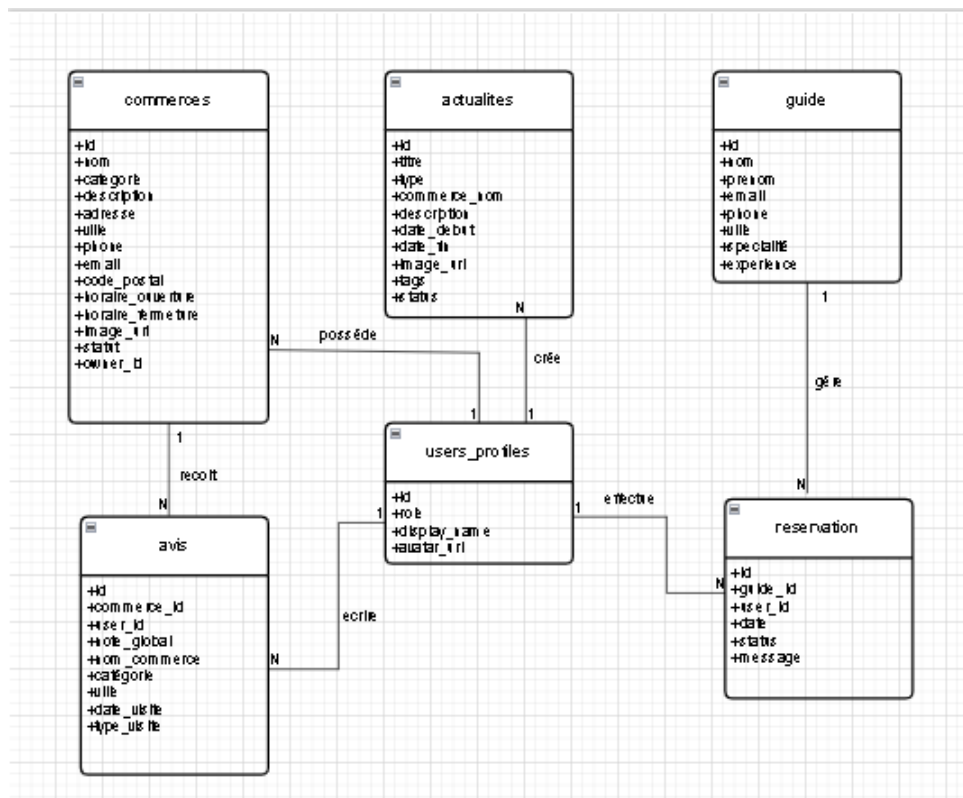


FIGURE 2.4 – diagramme de classe

2.5 Architecture technique

L'architecture du système repose sur une architecture en trois couches :

- **Frontend** : HTML / CSS / JavaScript
- **Backend** : Supabase (API + Auth + logique)
- **Base de données** : PostgreSQL (via Supabase)

Flux de données :

Frontend → API Supabase → Base de données

2.6 Maquettes / wireframes



FIGURE 2.5 – accueil

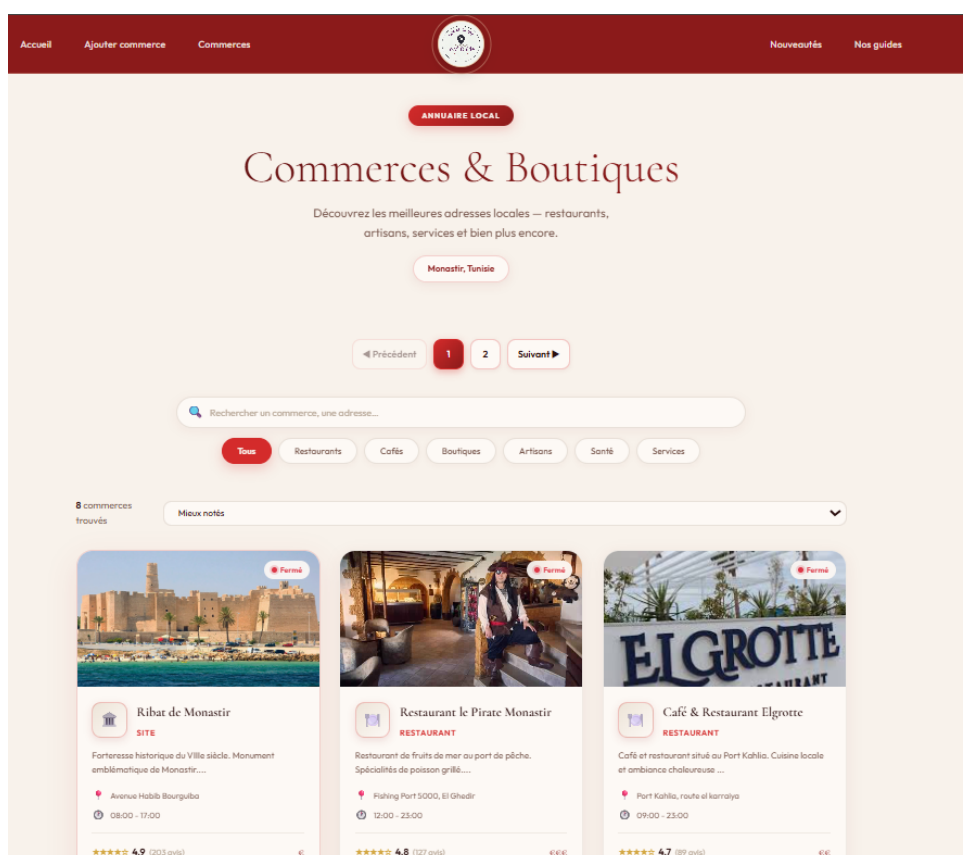


FIGURE 2.6 – commerce



FIGURE 2.7 – nouveauté



FIGURE 2.8 – guides

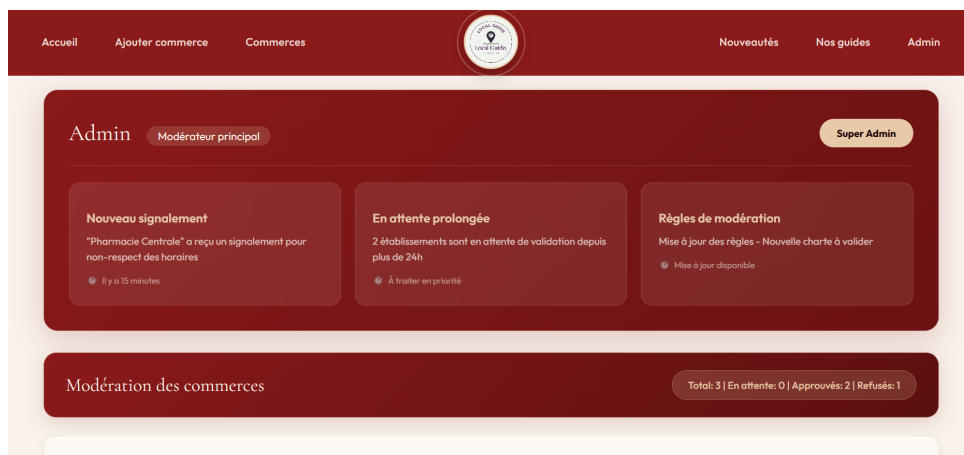


FIGURE 2.9 – Admin

AccueilAjouter commerceCommerces

NouveautésNos guidesAdmin

RÉFÉRENCER MON ÉTABLISSEMENT

Ajouter *votre* commerce

Rejoignez le réseau LocalGuide et touchez des milliers de visiteurs à Monastir

Informations générales

Présentez votre établissement

Nom du commerce *

Ex : Café El Bahri, Boulangerie Bled...

Catégorie *

Fourchette de prix

Choisir une catégorie

€

€€

€€€

Description

Décrivez votre commerce, votre ambiance, vos spécialités...

0 / 500 caractères

Coordonnées

Où vous trouver ?

Adresse *

Ex : Rue de la Liberté, Médina de Monastir

Ville *

Ex : Monastir

Code postal *

Ex : 5000

Téléphone *

Ex : +216 73 000 000

Email *

Ex : contact@moncommerce.tn

Horaires & Photo

Quand vous rendre visite ?

Source et numérisation

FIGURE 2.10 – ajout commerce

3 Développement

3.1 Technologies choisies et justifications

- **Front-end** : HTML5 / CSS3 / JS ES6
 - Légèreté, pas de build tool nécessaire, déploiement immédiat
- **BaaS** : Supabase
 - PostgreSQL managé + Auth + RLS + Realtime + SDK JS intégré
- **Base de données** : PostgreSQL 15
 - Robustesse, extensions PostGIS et pg_trgm, fonctions RPC
- **Recherche** : pg_trgm + tsvector
 - Recherche full-text en français avec tolérance aux fautes
- **Géolocalisation** : PostGIS
 - Requêtes de proximité spatiale (ST_DWithin)
- **Sécurité** : Row Level Security
 - Contrôle d'accès granulaire directement dans la base de données

3.2 Organisation du code

```
LocalGuide/

    .github/workflows/ci.yml

    backend/supabase/
        migrations/      (Scripts de migration DB)
        seeds/           (Donnees initiales)
        functions/       (Fonctions PostgreSQL)

    docs/
        rapport/         (Rapport du projet)
        specifications/  (Cahier des charges)

    frontend/
        css/             (Styles CSS)
        images/          (Images et logos)
        assets/          (Autres ressources)

    js/
        supabaseClient.js (Connexion Supabase)
        common.js         (Fonctions partagees)
        accueil.js        (Page d'accueil)
```

fichecomm.js	(Gestion commerces)
guide.js	(Reservation guides)
news.js	(Actualites)
admin.js	(Dashboard admin)
add_news.js	(Ajout actualites)
pages/	
aceuil.html	(Accueil)
fichecomm.html	(Annuaire commerces)
guide.html	(Liste guides)
news.html	(Actualites)
add.html	(Ajout commerce)
add_news.html	(Ajout actualite)
avis.html	(Deposer avis)
devenir_guide.html	(Candidature)
login.html	(Connexion)
Register.html	(Inscription)
admin.html	(Administration)
public/	
css/	(Styles publics)
images/	(Images publiques)
scripts/	
deploy.sh	(Script deploiement)
backup.sh	(Script sauvegarde)
src/	(Sources additionnelles)
supabase/	
config.toml	(Configuration)
seed.sql	(Donnees de test)
tests/	
unit/	(Tests unitaires)
integration/	(Tests integration)
.env.example	(Variables d'environnement)
.gitignore	(Fichiers ignores Git)
arborescence.txt	(Arborescence generee)
index .html	(Page principale)
package-lock.json	(Verrouillage versions)
package.json	(Dependances Node.js)

3.3 Implémentation des fonctionnalités clés

3.3.1 Authentification et gestion des rôles

La connexion utilise le SDK Supabase Auth. Après authentification, le rôle de l'utilisateur est lu depuis la table `users_profiles` pour adapter la navigation (affichage conditionnel du lien Admin).

Listing 3.1 – Connexion et détection du rôle (login.html)

```
1
2 const { data, error } = await supabase.auth.signInWithPassword({
3   email: email,
4   password: password
5 })
6
7 if (error) {
8   messageDiv.innerHTML = 'erreur ' + error.message
9   messageDiv.className = 'message error'
10  return
11 }
12
13 // Déterminer le rôle
14 let role = 'client'
15 const userEmail = data.user.email
16
17 // FORCER ADMIN POUR CET EMAIL
18 if (userEmail === 'admin@localguide.com') {
19   role = 'admin'
20   console.log('Admin détecté !')
21 } else {
22   // Vérifier dans users_profiles
23   const { data: profile } = await supabase
24     .from('users_profiles')
25     .select('role')
26     .eq('id', data.user.id)
27     .single()
28
29   if (profile && profile.role) {
30     role = profile.role
31   }
32 }
33
34 messageDiv.innerHTML = `Connect en tant que ${role} !
35   Redirection...`
36 messageDiv.className = 'message success'
37
38 // Rediriger
39 setTimeout(() => {
40   if (role === 'admin') {
41     window.location.href = 'admin.html'
42   } else {
43     window.location.href = 'accueil.html'
```

```

43     }
44     }, 1000)

```

3.3.2 Recherche de commerces

Listing 3.2 – Rechercher des commerces (fichecomm.html)

```

1
2 let liste = [...allCommerces]
3
4 // Filtre par cat gorie
5 if (activeFilter !== "all") {
6     liste = liste.filter(c => c.categorie === activeFilter)
7 }
8
9 // FILTRE DE RECHERCHE TEXTE
10 if (searchQuery.trim()) {
11     const q = searchQuery.toLowerCase()
12     liste = liste.filter(c =>
13         c.nom.toLowerCase().includes(q) ||
14         c.description?.toLowerCase().includes(q)
15     )
16 }
17 // Tri
18 if (sortBy === "note") liste.sort((a, b) => b.note - a.note)
19 if (sortBy === "nom") liste.sort((a, b) => a.nom.localeCompare(b.
    nom))
20
21 displayedCommerces = liste
22 currentPage = 1
23 hasMore = displayedCommerces.length > itemsPerPage
24
25 const countDisplay = document.getElementById("countDisplay")
26 if (countDisplay) countDisplay.textContent = displayedCommerces.
    length
27
28 renderCurrentPage()
29 renderPaginationButtons()
30 }

```

3.3.3 Gestion des guides

Listing 3.3 – exemple de code de gestion guides(guide.js)

```

1 async function loadGuidesEnAttente() {
2     const { data, error } = await supabase
3         .from('guides')
4         .select('*')
5         .eq('statut', 'inactif')
6         .order('created_at', { ascending: false })

```

```

7
8     if (error) {
9         console.error('Erreur chargement guides:', error)
10        return []
11    }
12    return data
13 }
14
15 async function approveGuide(id) {
16     if (!confirm(' Approuver ce guide ? Il sera visible sur la page
17         des guides. ')) return
18
19     const { error } = await supabase
20         .from('guides')
21         .update({
22             statut: 'disponible',
23             updated_at: new Date()
24         })
25         .eq('id', id)
26
27     if (error) {
28         alert(' Erreur: ' + error.message)
29     } else {
30         alert('Guide approuv ! Il est maintenant visible sur le
31             site.')
32         await renderGuidesEnAttente()
33     }
34 }

```

3.3.4 Upload de la photo du guide

```

1 async function handleFile(file) {
2     const error = document.getElementById("pictureError");
3     const maxSize = 5 * 1024 * 1024;
4     const allowed = ["image/jpeg", "image/png", "image/webp"];
5
6     if (!allowed.includes(file.type)) {
7         showError(error, "Format non support . Utilisez JPG, PNG ou
8             WEBP.");
9         return;
10    }
11
12    if (file.size > maxSize) {
13        showError(error, 'Fichier trop lourd (max 5MB).');
14        return;
15    }
16
17    try {
18        const fileExt = file.name.split('.').pop();

```

```

18     const fileName = `guide_${Date.now()}_${Math.random().toString
      (36).substring(7)}.${fileExt}`;
19
20     // Upload vers bucket 'guides-photos'
21     const { error: uploadError } = await supabase.storage
22       .from('guides-photos')
23       .upload(fileName, file)
24
25     if (uploadError) throw uploadError
26
27     // Récupérer l'URL publique
28     const { data: { publicUrl } } = supabase.storage
29       .from('guides-photos')
30       .getPublicUrl(fileName)
31
32     uploadedImageUrl = publicUrl
33
34   } catch (err) {
35     console.error('Erreur:', err);
36     showError(error, "Erreur inattendue lors de l'upload");
37   }
38 }

```

3.3.5 Sécurité Row Level Security

Toutes les tables sont protégées par des politiques RLS strictes :

```

1  -- Tout le monde peut lire les guides
2  CREATE POLICY "public_view_guides"
3    ON public.guides FOR SELECT TO public USING (true);
4
5  -- Seul le propriétaire ou un admin peut modifier
6  CREATE POLICY "owner_or_admin_update_guides"
7    ON public.guides FOR UPDATE TO authenticated
8    USING (auth.uid() = user_id OR auth.uid() IN (
9      SELECT user_id FROM public.user_roles WHERE role = 'admin'
10    ));

```

Listing 3.4 – Exemple de politiques RLS sur la table guides

4 Tests et validation

4.1 Environnement et outils de test

Les tests ont été effectués dans l'environnement suivant :

- **Navigateurs testés** : Google Chrome (v124), Brave et le navigateur intégré d'Android.
- **Base de données** : Console Supabase (éditeur SQL intégré) pour la vérification des requêtes et des politiques RLS (*Row Level Security*).
- **Tests d'interface** : Tests manuels des formulaires de connexion, d'inscription, d'ajout de commerce et de réservation.
- **Validation des données** : Vérification des contraintes NOT NULL, des types et des clés étrangères directement via Supabase.
- **Tests géographiques** : Vérification de la localisation des commerces et des guides sur carte (coordonnées latitude/longitude).

4.2 Scénarios de test

Fonctionnalité	Statut	Observations
Inscription utilisateur	OK	Création du profil dans <code>users_profiles</code>
Connexion utilisateur	OK	Authentification via Supabase Auth
Navigation sans connexion (visiteur)	OK	Accès en lecture seule aux commerces
Ajout d'un commerce (guide)	OK	Statut initial <code>en_attente</code>
Validation d'un commerce (admin)	OK	Passage du statut à approuvé
Refus d'un commerce avec motif (admin)	OK	Champ <code>raison_refus</code> renseigné
Recherche de commerces par nom	OK	Recherche insensible à la casse
Affichage des commerces sur carte	OK	Coordonnées GPS correctement récupérées
Dépôt d'un avis sur un commerce	OK	Note globale et notes détaillées enregistrées
Réservation auprès d'un guide	OK	Statut initial <code>en_attente</code>
Confirmation de réservation (guide)	OK	Statut mis à jour en confirmée
Annulation de réservation	OK	Statut mis à jour en annulée
Publication d'une actualité	OK	Statut publié après validation
Affichage du profil guide	OK	Informations et spécialités affichées
Gestion des rôles (admin)	OK	Attribution correcte via <code>user_roles</code>

5 Conclusion et perspectives

5.1 Bilan du projet

Le projet a permis de développer une application complète intégrant frontend et backend réel, de manipuler une base de données dynamique.

5.2 Difficultés rencontrées et apprentissages

- **Row Level Security** : La mise en place des politiques RLS a demandé une compréhension fine des rôles Supabase et de la différence entre `TO public` et `TO authenticated`.
- **Trigger de création de profil** : La synchronisation entre `auth.users` et `users_profiles` via trigger PostgreSQL a nécessité plusieurs itérations pour gérer les cas d'erreur.
- **PostGIS** : L'activation de l'extension et la compréhension des types `GEOMETRY(Point, 4326)` ont représenté une courbe d'apprentissage non négligeable.
- **Modules ES6** : La gestion des imports JavaScript entre fichiers partagés (`common.js`, `supabaseClient.js`) en mode module a requis une organisation rigoureuse.

5.3 Évolutions possibles

- **Application mobile** : portage en Progressive Web App (PWA) ou application React Native.
- **Système de notifications** : push notifications via Supabase Realtime pour les nouvelles réservations et validations.
- **Paiement en ligne** : intégration d'une passerelle de paiement (Stripe ou solution locale tunisienne) pour les réservations guides.
- **Carte interactive** : affichage cartographique des commerces et guides via Leaflet.js en exploitant les données PostGIS.
- **Extension géographique** : déploiement de la plateforme à d'autres villes tunisiennes (Sousse, Sfax, Tunis).
- **Tableau de bord analytique** : statistiques de fréquentation, notes moyennes, tendances de réservation pour les commerçants et guides.

6 Annexes

6.1 Lien vers le dépôt Git

<https://github.com/Souha-05/local>

Captures d'écran



FIGURE 6.1 – Page d'accueil LocalGuide — hero Monastir



FIGURE 6.2 – Tableau de bord administrateur — modération commerces



FIGURE 6.3 – Page guides — cartes avec filtres par spécialité

6.1.1 Fichiers de configuration

Configuration du client Supabase

Le fichier de configuration principal pour la connexion à Supabase est présenté ci-dessous. Il utilise les variables d'environnement pour sécuriser les identifiants.

```

1 const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
2 const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY
3
4 import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2/+esm'
5
6 export const supabase = createClient(supabaseUrl, supabaseAnonKey)
7
8 console.log(' Supabase connect ')
```

Listing 6.1 – supabase.js - Connexion à Supabase

Fichier package.json

Le fichier `package.json` définit les dépendances et les scripts nécessaires au lancement de l'application front-end.

```

1 {
2   "name": "web",
3   "version": "0.0.0",
4   "private": true,
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "preview": "vite preview"
10  },
11  "devDependencies": {
12    "vite": "^8.0.8"
```

```
13 }  
14 }
```

Listing 6.2 – package.json - Configuration Vite

6.1.2 Fichier d’environnement (.env)

Les identifiants de connexion à Supabase sont stockés dans un fichier `.env` à la racine du projet (non versionné pour des raisons de sécurité) :

```
1 VITE_SUPABASE_URL=https://auumbkcfwjpvdifnxyzly.supabase.co  
2 VITE_SUPABASE_ANON_KEY=sb_publishable_WXvPDV94YwVHQGAuVK0Nqg_Z3rIk1it
```

Listing 6.3 – .env - Variables d’environnement